

MIAUTRIX LICENSING CLIENT INTEGRATION GUIDE

PURPOSE

MiautrixLicensingClient.dll is the .NET client library for activating and checking in licenses against the Miautrix Licensing API.

Live API base URL:

<https://licensing-api.miautrix.tech>

Endpoints used by the DLL:

POST /api/activate

POST /api/checkin

GET /health

REQUIREMENTS

- .NET 10 runtime for apps targeting the current DLL build.
- Windows for the built-in HardwareFingerprint implementation because it uses WMI and System.Management.
- Internet access to <https://licensing-api.miautrix.tech>.
- No private key, AES key, database connection string, or admin credential is required in the client.

FILES TO DISTRIBUTE

For a direct DLL integration, provide:

- MiautrixLicensingClient.dll
- MiautrixLicensingClient.deps.json when publishing framework-dependent apps
- Matching .pdb only if debug symbols are desired

Recommended path for integrators is to reference the project/package produced from MiautrixLicensingClient rather than copying source files.

BASIC ACTIVATION FLOW

1. Generate and persist one InstallGuid per installed app instance.
2. Read hardware with HardwareFingerprint.Read() or provide equivalent stable values.
3. Call ActivationApiClient.ActivateAsync().
4. If the response contains LicenseFileBase64, verify it with LicenseFile.VerifyAndParse().
5. Persist the returned ActivationId, InstallGuid, and license file.
6. Periodically call CheckinAsync() using the returned policy interval.

ACTIVATION BEHAVIOR

- Same hardware reactivation reuses the existing activation row.

- New hardware auto-approves while approved activations are below `License.MaxActivations`.
- New hardware over the approved activation limit returns `PendingReview`.
- Test licenses bypass activation limits.
- Revoked and expired licenses fail activation.
- Optional activation email updates the license customer email server-side.

C# EXAMPLE

```
using System.Security.Cryptography;
using Miautrix.Licensing.Client;

var baseUrl = "https://licensing-api.miautrix.tech";
var licenseKey = "XXXX-XXXXX-XXXX-XXXX-XXXX-XXXX-XX";
var installGuid = LoadOrCreateInstallGuid();
var hardware = HardwareFingerprint.Read();

using var client = new ActivationApiClient(baseUrl);
var (httpStatus, result) = await client.ActivateAsync(new ActivateRequest
{
    LicenseKey = licenseKey,
    InstallGuid = installGuid,
    Hardware = hardware,
    AppVersion = "1.0.0",
    Email = "customer@example.com",
    VersionId = null,
    ClientTimestampUtc = DateTime.UtcNow,
});

if (!result.Success)
{
    throw new InvalidOperationException($"Activation failed: HTTP {httpStatus},
{result.Code}, {result.Message}");
}

if (result.Data?.LicenseFileBase64 is not { Length: > 0 } licenseFileBase64)
{
    throw new InvalidOperationException("Activation did not return a license file.");
}

using var publicKey = RSA.Create();
publicKey.ImportFromPem(Keys.LicensingPublicKeyPem);
var file = LicenseFile.VerifyAndParse(licenseFileBase64, publicKey);

if (!file.SignatureVerified)
{
    throw new InvalidOperationException("License file signature verification failed.");
}

SaveActivationState(result.Data.ActivationId, installGuid, licenseFileBase64);
```

CHECK-IN EXAMPLE

```
var (httpStatus, result) = await client.CheckinAsync(new CheckinRequest
{
    ActivationId = savedActivationId,
    InstallGuid = savedInstallGuid,
    Hardware = HardwareFingerprint.Read(),
    LastLocalStatus = savedStatus,
    ClientTimestampUtc = DateTime.UtcNow,
});

if (result.Code == ResultCode.Locked)
{
    // Stop trusting any cached license file.
    DisableLicensedFeatures(result.Data?.Reason);
}
else if (result.Success && result.Data?.LicenseFileBase64 is { Length: > 0 }
refreshedFile)
{
    // Verify before storing/trusting.
    var parsed = LicenseFile.VerifyAndParse(refreshedFile, publicKey);
    if (parsed.SignatureVerified)
        SaveLicenseFile(refreshedFile);
}
```

HARDWARE FINGERPRINT RULES

The API identifies a machine by exact match on these four fields:

- CpuId
- MotherboardSerial
- TpmId
- MacAddressPrimary

If a value cannot be read, use a stable placeholder. Do not send random or changing values.

PRODUCT VERSIONS

ActivateRequest.VersionId is optional. If supplied, it must match the version stored on the license. If it does not match, the API returns InvalidKeyFormat.

EMAIL/CUSTOMER UPDATE

ActivateRequest.Email is optional. When provided, the API trims it and stores it on License.CustomerEmail. This is useful for associating a license with the activating customer during field activation.

LICENSE FILE VERIFICATION

The returned LicenseFileBase64 is base64-encoded signed XML. It is not encrypted.

Verification uses:

- RSA-SHA256
- PKCS#1 v1.5 padding
- Public key embedded in Keys.LicensingPublicKeyPem

Always verify the signature before trusting status, expiry, activation ID, or hardware data.

COMMON RESULTS

ResultCode	Meaning
---	---
Activated	Activation is approved.
PendingReview	Activation exists but requires admin review.
Renewed	Check-in succeeded.
Locked	Check-in succeeded but local license must stop being trusted. Check Reason.
InvalidKeyFormat	Bad key format or product version mismatch.
LicenseNotFound	Key does not exist.
LicenseExpired	License is expired.
LicenseRevoked	License is revoked.
RateLimited	Too many requests from the same IP.
ServerError	Unexpected or unparseable server response.

TROUBLESHOOTING

CLOUDFLARE 502 OR NON-JSON RESPONSE

Older clients could throw JSON parse exceptions. Current DLL maps non-JSON response bodies to ServerError with the raw message. Check API service status and runtime installation on the server.

PENDINGREVIEW

This is expected when a new hardware fingerprint exceeds the approved activation limit. The signed license file status will be pending_review and includes a review deadline.

LICENSEREVOKED

The license was revoked in the admin panel. Activation should fail and no local cached license should be trusted.

VERSION MISMATCH

If VersionId is sent and does not match the license version, activation fails with InvalidKeyFormat. Omit VersionId for backwards-compatible clients unless the app knows the exact version ID.

SECURITY NOTES

- The RSA public key is safe to embed.
- Never embed server private keys, database credentials, admin credentials, or connection strings.
- Do not skip signature verification.
- Treat unknown result codes as failures.